

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #12 - Semana 13 – Sivarama - Capítulo 21
Interfaz de lenguaje de alto nivel

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 22/10/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 423–441.

Abstract—Mixed-mode programming involves writing parts of a program in different languages to leverage the advantages of each. By combining C and assembly, high-level code provides productivity and portability, while low-level code offers hardware access and performance optimization. There are two main methods for mixed-mode programming: inline assembly, where assembly instructions are embedded within C code, and separate modules, where assembly code is compiled independently. Communication between C and assembly relies on the stack, with conventions for passing parameters and returning values. The AT&T syntax used by gcc is also covered, highlighting differences from Intel syntax.

I. Lenguaje de alto nivel y programación en modo mixto

La programación en modo mixto implica escribir partes de un programa en diferentes lenguajes de programación. En este capítulo, se aborda la integración de lenguajes C y ensamblador. Se explican dos métodos: código ensamblador en línea dentro de C o módulos ensamblador separados, y cómo se combinan ambos para generar programas eficientes.

A. Introducción a la programación en modo mixto

Se utiliza la programación en modo mixto para aprovechar las ventajas de ambos lenguajes. C, al ser de alto nivel, permite mayor productividad y portabilidad, mientras que ensamblador proporciona acceso a bajo nivel y optimización de rendimiento. Esto resulta útil en software de sistema, donde se pueden mezclar funciones de alto nivel con ensamblador para obtener acceso directo al hardware o mejorar tiempos de ejecución.

B. Desventajas del ensamblador puro

Aunque es posible programar completamente en ensamblador, tiene varias desventajas:

- Baja productividad: debido a la cantidad de instrucciones necesarias para realizar tareas simples.
- Alto costo de mantenimiento: los programas en ensamblador son difíciles de mantener.
- Falta de portabilidad: los programas no son fácilmente transferibles entre plataformas.

C. Ventajas del ensamblador

A pesar de las desventajas, ensamblador ofrece ventajas como el acceso a hardware y la posibilidad de optimizar el tiempo de ejecución. Por lo tanto, la programación en modo mixto combina lo mejor de ambos mundos.

II. Métodos de programación en modo mixto

Existen dos formas principales de combinar código en C y ensamblador:

A. Ensamblador en línea

El ensamblador en línea permite insertar instrucciones de ensamblador directamente dentro de un programa en C. Para ello, se utilizan constructos como `asm` que permiten al compilador distinguir entre las instrucciones en C y las de ensamblador. Este método es adecuado cuando se necesita una pequeña cantidad de ensamblador en el programa.

B. Módulos de ensamblador separados

Para cantidades mayores de código ensamblador, es preferible usar módulos de ensamblador separados. Estos módulos se compilan de manera independiente y luego se enlazan con los módulos en C usando el enlazador. Se compilan los módulos C con `gcc` y los módulos ensamblador con `nasm`, como se muestra en el siguiente comando:

```
nasm -f elf sample2.asm
```

```
gcc -o output sample1.c sample2.o
```

III. Llamadas entre C y ensamblador

A. Llamadas de procedimientos de ensamblador desde C

Cuando se llama a un procedimiento en ensamblador desde C, el mecanismo de comunicación se realiza a través de la pila. En C, los argumentos se empujan en la pila de derecha a izquierda. El código ensamblador se encarga de procesar estos argumentos y devolver el resultado en el registro `EAX`. Un ejemplo simple en C sería:

```
extern int test(int, int, int);
value = test(x, y, 5);
```

Y su traducción a ensamblador sería:

```
push 5
push 70
push 25
call test
add ESP, 12
mov [EBP-12], EAX
```

B. Llamadas de funciones C desde ensamblador

También es posible llamar a funciones en C desde ensamblador. En este caso, se utilizan las mismas técnicas de manejo de pila para pasar argumentos a las funciones como printf o scanf. Por ejemplo, para imprimir un mensaje y leer valores desde el ensamblador, se haría lo siguiente:

```
push ECX          ; Tamaño del array
push mensaje      ; Mensaje
call printf
```

IV. Sintaxis de AT&T vs Intel

A. Diferencias clave

La sintaxis de AT&T, utilizada por gcc, difiere de la sintaxis de Intel. Algunas de las diferencias clave son:

1. **Nombres de registros:** En AT&T, los nombres de registros se prefijan con % (ejemplo: %eax).
2. **Orden de operandos:** En AT&T, el orden es inverso; el origen va primero y el destino después (ejemplo: mov %ebx, %eax).
3. **Sufijos de tamaño de operandos:** Los tamaños de los operandos se indican con sufijos como b (byte), w (word), y l (longword).
4. **Operandos inmediatos:** Se prefijan con \$ (ejemplo: movl \$5, %eax).

B. Declaraciones simples y extendidas en ensamblador en línea

Una declaración simple de ensamblador en línea puede ser:

```
asm("incl %eax");
```

Mientras que las declaraciones extendidas permiten especificar entradas, salidas y registros modificados:

```
asm("movl %1, %0" : "=r" (sum) : "r" (number1));
```

Aquí, sum recibe el valor de number1 utilizando registros especificados por el compilador.

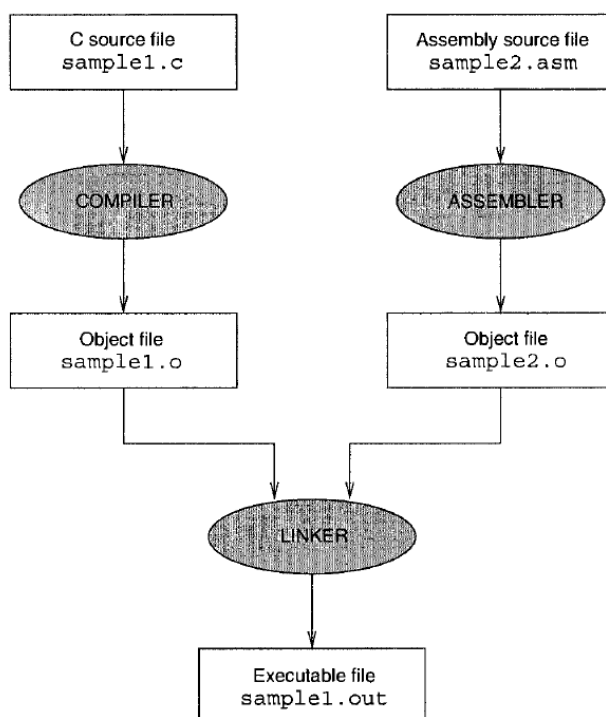


Figura 1. Pasos para compilar programas de modo mixto